Simulacro 1 · Biblioteca del instituto

Acceso a Datos · 2.º DAM · Duración: 120 minutos · Formato igual que el repaso UT1 (con Main)

Gestión de libros

El instituto registra sus libros en un fichero CSV. Desarrolla una aplicación Java por consola que **recupere los libros**, **conserve una copia binaria** y **genere un JSON con los disponibles**. Trabaja individualmente.

Se proporciona un proyecto Maven con Gson configurado, **Main** y los DAO por implementar, la carpeta **datos** y la clase **Libro** con constructor, getters, setters y toString(). Reserva los primeros 10 minutos para leer el enunciado y abrir el proyecto.

Atributos de Libro: `idLibro` (int), `titulo` (String), `autor` (String), `precio` (double) y `disponible` (boolean). El identificador y el precio deben ser mayores que cero.

El fichero `datos/libros.csv` contiene:

```
idLibro;titulo;autor;precio;disponible
1;El Quijote;Cervantes;12.50;true
2;La Celestina;Rojas;9.90;false
3;Rimas;Becquer;7.00;true
4;Fortunata y Jacinta;Galdos;15.25;false
5;La Regenta;Clarin;14.00;true
```

Los campos no contienen punto y coma ni saltos de línea. Los decimales usan punto. La carpeta datos ya existe.

1 Recuperar los libros desde CSV — 25 min | 2 pts

Implementa en **LibroDAOCSV** el método `cargar(String ruta)`, que devuelve `ArrayList<Libro>` y puede propagar `IOException`. Omite la cabecera y convierte los registros en objetos.

Comprueba que cada registro tenga **cinco campos**, identificador entero positivo, **precio decimal positivo** y disponible igual a true o false. Si una línea es incorrecta, informa de su número y motivo, omítela y continúa. Los fallos de apertura o lectura se propagarán a Main.

> **PISTA** Para leer un decimal usa `Double.parseDouble`. Si el texto no es un número, lanza `NumberFormatException`, igual que parseInt.

2 Persistencia binaria — 20 min | 2 pts

Implementa en **LibroDAOBinario** los métodos `guardar(String ruta, ArrayList<Libro> libros)` y `cargar(String ruta)`, que devuelve `ArrayList<Libro>`. Utiliza `ObjectOutputStream` y `ObjectInputStream` y adapta el modelo para permitir la serialización.

Guarda la colección completa en **datos/libros.dat** y recupérala. Cada guardado sustituirá el contenido anterior. Propaga `IOException` y, en la lectura, `ClassNotFoundException`.

3 Registrar un préstamo y exportar a JSON — 20 min | 2 pts

Sobre la colección recuperada del binario, localiza el **libro 5** y márcalo como **no disponible** (se lo han llevado prestado). Si no existe, informa de ello.

Implementa `LibroDAOJSON.guardar(String ruta, ArrayList<Libro> libros)` con Gson, propagando `IOException`. Selecciona los libros **disponibles** y guárdalos como array JSON en **datos/disponibles.json**. Cada exportación debe sustituir el contenido anterior.

4 Integración y excepciones — 25 min | 2 pts

Completa Main para ejecutar, en este orden:

1. Cargar datos/libros.csv.

2. Guardar la colección completa en datos/libros.dat y recuperarla desde ese fichero.

3. Registrar el préstamo del libro 5 en la colección recuperada.

4. Volver a guardar la colección actualizada en el binario.

5. Seleccionar los libros disponibles y exportarlos a datos/disponibles.json.

6. Mostrar el número de libros cargados, recuperados y exportados.

Mantén la lectura y escritura en los DAO. Realiza la búsqueda, modificación y selección en Main. Gestiona las excepciones desde Main con mensajes claros; garantiza el cierre de los recursos y no continúes con fases dependientes si falla una operación.

Con el CSV inicial deben cargarse **5 libros** y exportarse **2 disponibles**, con los identificadores **1 y 3**. El binario debe conservar el libro 5 como no disponible. El CSV original no se modifica.

5 Pruebas y documentación — 20 min | 2 pts

Realiza estas pruebas sobre **copias independientes** del CSV original y anota el resultado obtenido:

Prueba	Resultado esperado
CSV original	Se cargan 5 libros y se exportan los disponibles 1 y 3.
`precio` con valor **gratis** en una fila	Se informa del error y se procesan las demás filas.
`precio` con valor **-4.5** en una fila	Se rechaza ese registro por precio no positivo.
`disponible` con valor **si** en una fila	Se rechaza ese registro por booleano incorrecto.
CSV inexistente	Se informa del error y no se continúa con la conversión.
Leer el binario tras registrar el préstamo	El libro 5 aparece como no disponible.
Ejecutar dos veces con el CSV original	No hay duplicados; se exportan 2 disponibles en ambas ejecuciones.

Entrega un **README.md** breve con instrucciones de ejecución y una tabla de pruebas con resultado esperado y obtenido. Documenta el método de carga CSV: qué devuelve y cómo trata los registros incorrectos.

R Reflexión final (para el README) — - min | extra pts

Imagina que, en `guardar`, abrieras el fichero en modo añadir: `new FileOutputStream(ruta, true)`. Si ejecutas el programa dos veces, ¿qué libros recuperarías al leer el binario con un único `readObject()`? Razona sin ejecutarlo.

> **PISTA** Cada vez que se crea un ObjectOutputStream se escribe una cabecera al principio. Piensa qué hay dentro del fichero tras la segunda ejecución y qué lee un solo readObject().

SOLUCIÓN ORIENTATIVA

OJO No mires esta parte hasta haber terminado el examen o hasta que pasen los 120 minutos. Hay muchas formas válidas de resolverlo: compara la idea, no letra por letra.

Apartado 1: LibroDAOCSV

Es el mismo esqueleto que tu **PrestamoDAOCSV**. Lo único que cambia es el precio (double) y que no hay que mirar nada especial con el booleano salvo comprobar true/false.

```
public class LibroDAOCSV {
    public ArrayList<Libro> cargar(String ruta) throws IOException {
        ArrayList<Libro> lista = new ArrayList<>();
        try (BufferedReader br = new BufferedReader(new FileReader(ruta))) {
            String linea;
            int numLinea = 1;
            br.readLine();                                  // cabecera
            while ((linea = br.readLine()) != null) {
                numLinea++;
                if (linea.trim().isEmpty()) {
                    System.err.println("Línea " + numLinea + ": está vacía");
                    continue;
                }
                String[] d = linea.split(";");
                if (d.length != 5) {
                    System.err.println("Línea " + numLinea + ": debe tener 5 campos y tiene "
                            + d.length);
                    continue;
                }
                try {
                    int id = Integer.parseInt(d[0].trim());
                    String titulo = d[1].trim();
                    String autor = d[2].trim();
                    double precio = Double.parseDouble(d[3].trim());
                    String txt = d[4].trim();

                    if (id <= 0) {
                        System.err.println("Línea " + numLinea + ": el id debe ser positivo");
                        continue;
                    }
                    if (precio <= 0) {
                        System.err.println("Línea " + numLinea + ": precio no positivo");
                        continue;
                    }
                    if (!txt.equals("true") && !txt.equals("false")) {
                        System.err.println("Línea " + numLinea + ": disponible no es true/false");
                        continue;
                    }
                    lista.add(new Libro(id, titulo, autor, precio, Boolean.parseBoolean(txt)));
                } catch (NumberFormatException e) {
                    System.err.println("Línea " + numLinea + ": número no válido -> "
                            + e.getMessage());
                }
            }
        }
        return lista;
    }
}
```

Apartados 2 y 3: binario y JSON

Libro debe ser **Serializable** (con serialVersionUID). Los DAO son idénticos a los de Prestamo.

```
// LibroDAOBinario
public void guardar(String ruta, ArrayList<Libro> libros) throws IOException {
    try (ObjectOutputStream out = new ObjectOutputStream(new FileOutputStream(ruta))) {
        out.writeObject(libros);
```

```
    }
}
@SuppressWarnings("unchecked")
public ArrayList<Libro> cargar(String ruta) throws IOException, ClassNotFoundException {
    try (ObjectInputStream in = new ObjectInputStream(new FileInputStream(ruta))) {
        return (ArrayList<Libro>) in.readObject();
    }
}

// LibroDAOJSON
private Gson gson = new GsonBuilder().setPrettyPrinting().create();
public void guardar(String ruta, ArrayList<Libro> libros) throws IOException {
    try (BufferedWriter bw = new BufferedWriter(new FileWriter(ruta))) {
        gson.toJson(libros, bw);
    }
}
```

Apartado 4: Main (el centro del programa)

```
try {
    ArrayList<Libro> cargados = csv.cargar("datos/libros.csv");
    bin.guardar("datos/libros.dat", cargados);
    ArrayList<Libro> recuperados = bin.cargar("datos/libros.dat");

    boolean encontrado = false;                  // fuera del bucle
    for (Libro l : recuperados) {
        if (l.getIdLibro() == 5) {
            l.setDisponible(false);
            encontrado = true;
        }
    }
    if (!encontrado) {
        System.err.println("El libro 5 no existe");
    }
    bin.guardar("datos/libros.dat", recuperados);   // si no, el cambio se pierde

    ArrayList<Libro> disponibles = new ArrayList<>();
    for (Libro l : recuperados) {
        if (l.isDisponible()) {
            disponibles.add(l);
        }
    }
    json.guardar("datos/disponibles.json", disponibles);

    System.out.println("Cargados: " + cargados.size());        // 5
    System.out.println("Recuperados: " + recuperados.size());  // 5
    System.out.println("Exportados: " + disponibles.size());   // 2
} catch (IOException e) {
    System.err.println("Error de lectura/escritura: " + e.getMessage());
} catch (ClassNotFoundException e) {
    System.err.println("No se encontró la clase al leer el binario: " + e.getMessage());
}
```

Reflexión

Con el modo añadir, el fichero contendría **dos cabeceras** y dos colecciones seguidas (la de la primera ejecución y la de la segunda). Un solo **readObject()** lee únicamente la **primera**, es decir, recuperarías la colección antigua y parecería que el guardado no hace nada. Si intentaras leer un segundo objeto saltaría **StreamCorruptedException**, porque la segunda cabecera no es un objeto. Por eso para "sustituir" se abre el fichero **sin** el true.